

chap 5.1 ~ 5.2

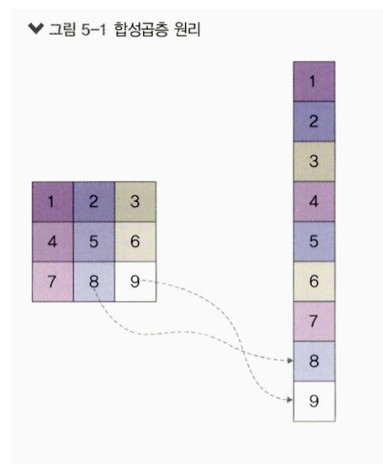
5장 합성곱 신경망 I

5.1 합성곱 신경망

순전파 과정에 따라 계산된 오차 정보는 신경망의 모든 노드(출력층→은닉층→입력층)로 전송된다. 이러한 계산 과정은 복잡하고 많은 자원(CPU 혹은 GPU, 메모리)을 요구하며 계산 시간도 오래 걸릴 수 있다. 이 문제를 해결하고자 하는 것이 **합성곱 신경망**이다. 합성곱 신경망은 이미지 전체를 한 번에 계산하지 않고 **이미지의 국소적 부분**을 계산함으로써 시간과 자원을 절약하며 **이미지의 세밀한 부분까지 분석**할 수 있다.

5.1.1 합성곱층의 필요성

합성곱 신경망은 이미지나 영상을 처리하는 데 유용하다. 예를 들어 3×3 흑백(그레이스케일) 이미지를 오른쪽과 같이 펼치면(플래튼) 각 픽셀의 값을 가중치를 곱하여 은닉층으로 전달한다. 그러나 이미지를 **평쳐서 분석**하면 데이터의 **공간적 구조**를 무시하게 되며, 이것을 방지하려고 도입된 것이 **합성곱층**이다.

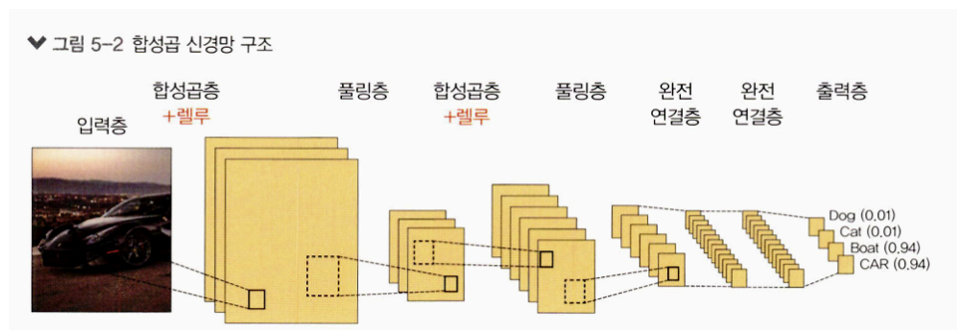


- 3×3 배열을 9×1 벡터로 펼치는 예를 통해, 평탄화 시 공간 구조 정보가 사라질 수 있음을 보인다.

5.1.2 합성곱 신경망 구조

합성곱 신경망(Convolutional Neural Network, CNN 또는 ConvNet)은 **음성 인식**이나 **이미지/영상 인식**에서 주로 사용되는 신경망이다. 다차원 배열 데이터를 처리하도록 구성되어 컬러 이미지 같이 **다차원 배열 처리**에 특화되어 있으며, 다음과 같이 계층 다섯 개로 구성된다.

1. 입력층
2. 합성곱층
3. 풀링층
4. 완전연결층
5. 출력층

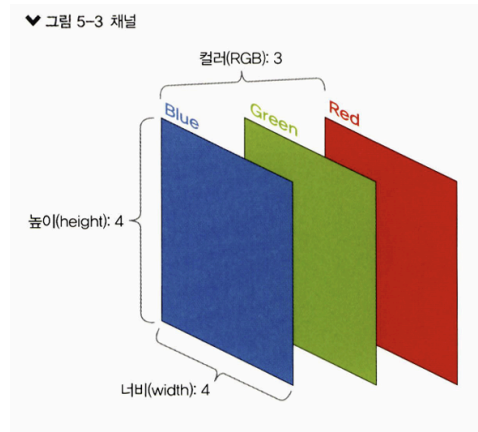


- 입력 → (합성곱층 + 풀링층) 반복 → 완전연결층 → 출력층(소프트맥스).
- 합성곱·풀링을 거치며 입력 이미지의 **주요 특성 벡터**를 추출하고, 완전연결층에서 1차원 벡터로 변환된 뒤 **소프트맥스(softmax)** 함수로 최종 결과가 출력된다.

입력층

입력층(input layer)은 입력 이미지 데이터가 최초로 거치게 되는 계층이다. 이미지는 단순 1차원이 아니라 **높이(height), 너비(width), 채널(channel)** 값을 갖는 **3차원 데이터**이다.

- 그레이스케일: 채널 수 1
- 컬러(RGB): 채널 수 3



- 예: 높이: 4, 너비: 4, 채널: 3(RGB) → 이미지 형태(shape)는 (4, 4, 3).

합성곱층

합성곱층(convolutional layer)은 입력 데이터에서 **특성을 추출**하는 역할을 수행한다. 이를 위해 **커널(=필터)** 을 사용한다. 커널/필터는 이미지의 모든 영역을 훑으면서 특성을 추출하며, 이렇게 추출된 결과물이 **특성 맵(feature map)** 이다.

- 커널 크기: **3×3**, **5×5**와 같은 크기가 일반적
- 스트라이드(stride)**: 지정된 간격에 따라 커널을 이동

스트라이드가 1일 때의 이동 과정 예시는 다음과 같다. 각 위치에서 **입력과 필터의 대응 원소 곱을 모두 더함**으로써 출력 값을 구한다.

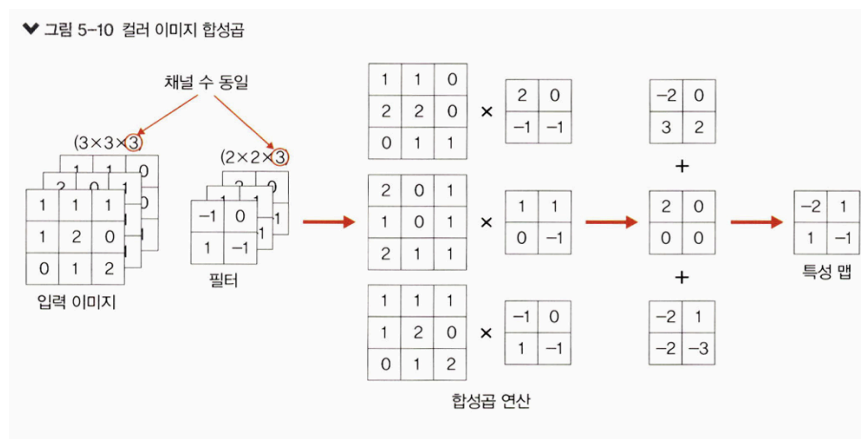


- 입력 이미지에 3×3 필터를 적용하고, 필터를 한 칸씩(스트라이드=1) 이동시키며 합성곱 연산을 반복하여 출력 **특성 맵**을 채운다.

앞선 예에서 이미지 크기가 (6, 6, 1)이고 3×3 커널, 스트라이드 1을 사용하면, 스트라이드 간격만큼 순회하여 모든 입력 값과의 합성곱 연산으로 **새로운 특성 맵**이 만들어지며, (6, 6, 1) 크기가 **(4, 4, 1)** 크기의 특성 맵으로 줄어든다.

컬러 이미지의 합성곱:

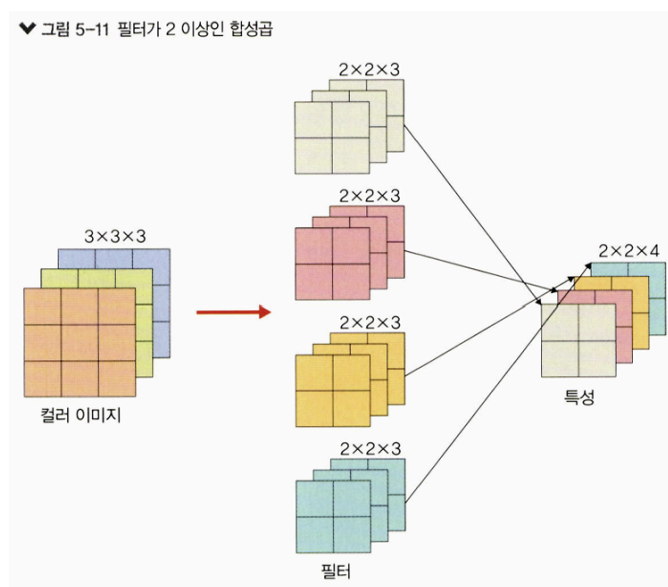
- 그레이스케일과의 차이점은 **채널 수가 3**이라는 점이며, **RGB 각 채널에 서로 다른 가중치로 합성곱**을 적용한 후 결과를 **더해** 특성 맵이 된다(스트라이드 및 연산 방법은 동일).



- 입력 이미지(3×3×3)와 동일한 채널 수를 갖는 필터(2×2×3)를 채널별로 합성곱한 뒤 더해 특성 맵을 얻는 과정을 보인다.

필터가 2개 이상인 경우:

- 필터 **개수만큼** 특성 맵이 생성되며, 각 필터가 다른 특성을 추출한다.



- 여러 필터($2 \times 2 \times 3$)를 사용하면 특성 맵의 채널 수가 필터 개수와 동일하게 증가한다.

합성곱층 요약:

- **입력 데이터:**

$$W_1 \times H_1 \times D_1 (\text{가로} \times \text{세로} \times \text{채널/깊이})$$

- **하이퍼파라미터**

- 필터 개수: K
- 필터 크기: F
- 스트라이드: S
- 패딩: P

- **출력 데이터**

$$W_2 = (W_1 - F + 2P) / S + 1$$

$$H_2 = (H_1 - F + 2P) / S + 1$$

$$D_2 = K$$

풀링층

풀링층(pooling layer)은 합성곱층과 유사하게 **특성 맵의 차원을 다운 샘플링**하여 연산량을 감소시키고, **주요한 특성 벡터**를 추출하여 학습을 효율적으로 할 수 있게 한다.

다운 샘플링



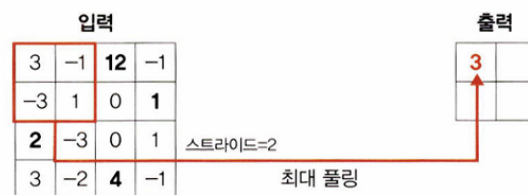
입력 이미지를 축소하는 예시.

풀링 연산은 두 가지가 사용된다.

- **최대 풀링(max pooling):** 대상 영역에서 **최댓값**을 추출
- **평균 풀링(average pooling):** 대상 영역에서 **평균**을 반환

대부분의 합성곱 신경망에서는 **최대 풀링**을 사용한다(평균 풀링은 각 커널 값을 평균화시켜 중요한 가중치를 갖는 값의 특성이 희미해질 수 있기 때문). 아래는 **최대 풀링**의 연산 과정이다(스트라이드=2).

▼ 그림 5-13 첫 번째 최대 풀링 과정



- 첫 번째 최대 풀링 과정: 3, -1, -3, 1 중 **최댓값 3** 선택 → 출력 첫 칸이 3.

▼ 그림 5-14 두 번째 최대 풀링 과정



- 두 번째 최대 풀링 과정: 12, -1, 0, 1 중 **최댓값 12** → 출력 두 번째 칸이 12.

▼ 그림 5-15 세 번째 최대 풀링 과정



- 세 번째 최대 풀링 과정: 2, -3, 3, -2 중 **최댓값 3** → 다음 행 첫 칸이 3.

♥ 그림 5-16 네 번째 최대 풀링 과정



- 네 번째 최대 풀링 과정: 0, 1, 4, -1 중 최댓값 4 → 다음 행 두 번째 칸이 4.

최대 풀링 결과, 출력 특성 맵은 위 네 값(3, 12, 3, 4)로 구성된다.

평균 풀링 계산식(최대 풀링과 동일한 위치에서 평균으로 계산)

$$0 = (3 + (-1) + (-3) + 1) / 4$$

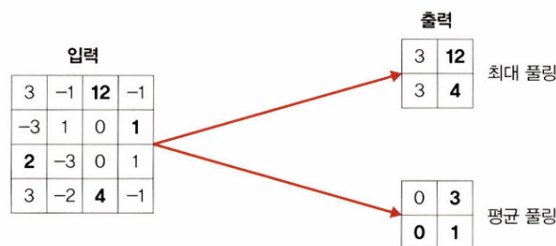
$$3 = (12 + (-1) + 0 + 1) / 4$$

$$0 = (2 + (-3) + 3 + (-2)) / 4$$

$$1 = (0 + 1 + 4 + (-1)) / 4$$

최대 풀링과 평균 풀링 비교

♥ 그림 5-17 최대 풀링과 평균 풀링 비교



- 동일 입력에 대해 최대 풀링 결과는 $[[3, 12], [3, 4]]$, 평균 풀링 결과는 $[[0, 3], [0, 1]]$ 로 제시됨.

풀링 요약(최대/평균 모두 동일한 파라미터 체계)

- 입력 데이터:

$$W_1 \times H_1 \times D_1$$

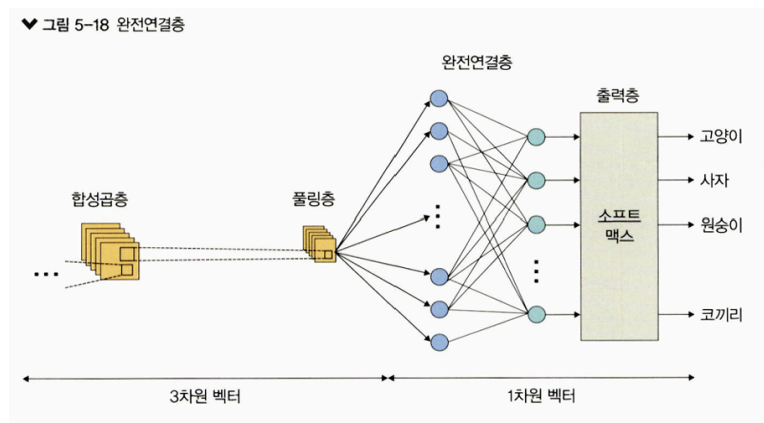
- 하이퍼파라미터
 - 필터 크기: F
 - 스트라이드: S
- 출력 데이터

$$W_2 = (W_1 - F) / S + 1$$

$$H_2 = (H_1 - F) / S + 1$$

$$D_2 = D_1$$

완전연결층



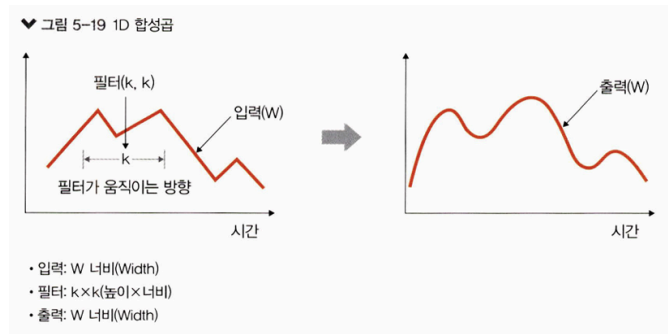
- 합성곱층과 풀링층을 거쳐 **차원이 축소된 특성 맵**이 **완전연결층(fully connected layer)** 으로 전달됨.
- 이 과정에서 이미지는 **3차원 벡터 → 1차원 벡터**로 **플래튼(flatten)** 됨.

출력층

- **출력층(output layer)**에서는 **소프트맥스 활성화 함수** 사용.
- 입력값을 0~1 사이의 값으로 출력하며, 소프트맥스 결과 중 **가장 높은 확률**을 갖는 레이블이 최종 값으로 선택됨.

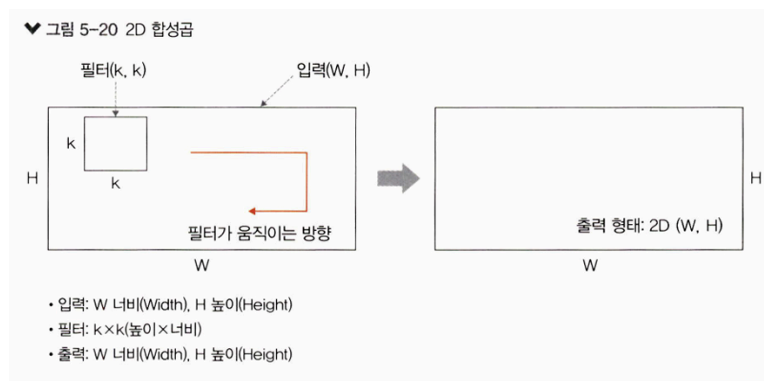
5.1.3 1D, 2D, 3D 합성곱

1D 합성곱



- 필터가 **시간 축으로만 이동**.
- 그래프 곡선을 완화할 때 많이 사용됨.
- 입력 W 와 필터 (k,k) $\rightarrow$ 출력 W (1D 배열).
 - 입력: W 너비(Width)
 - 필터: $k * k$ (높이) * 너비
 - 출력: W 너비(Width)

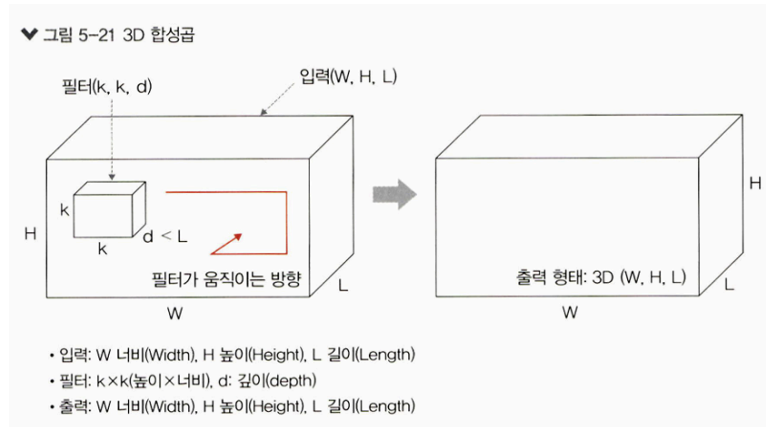
2D 합성곱



- 필터가 **가로·세로 두 방향으로 이동**.
- 입력 (W,H), 필터 (k,k) $\rightarrow$ 출력 (W,H)(2D 행렬).
 - 입력: W 너비, H 높이
 - 필터: $k * k$ (높이) * 너비

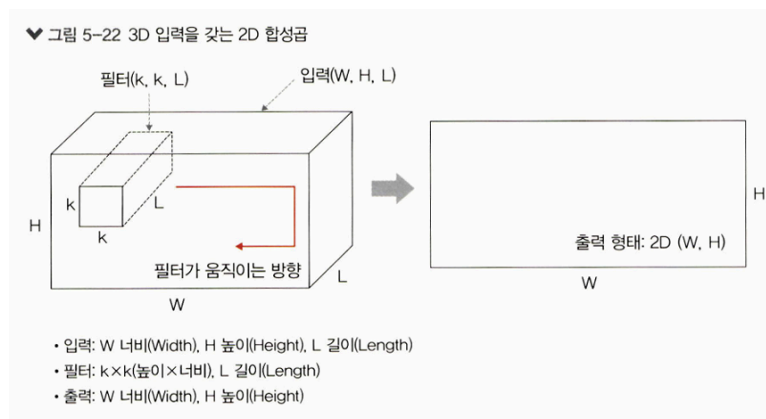
- 출력: 2D (W,H)

3D 합성곱



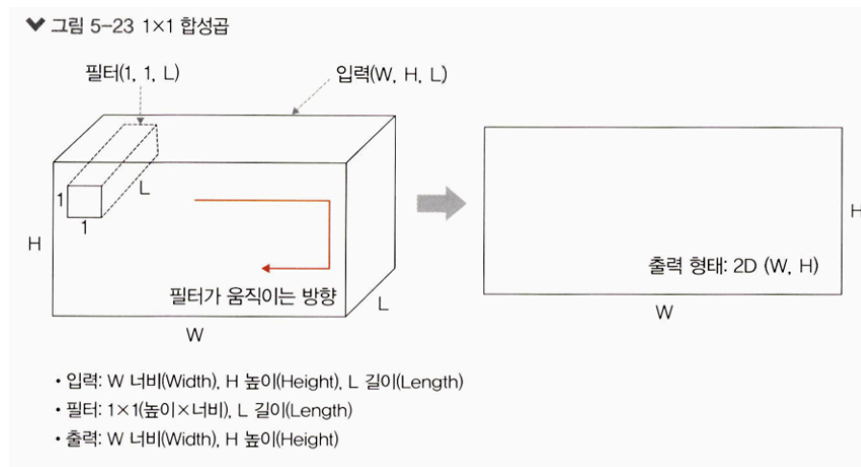
- 필터가 **세 방향**으로 이동.
- 입력 (W,H,L), 필터 (k,k,d) → 출력 (W,H,L)(3D 형상).
- **중요:** 이때 $d < L$ 유지.

3D 입력을 갖는 2D 합성곱



- 입력이 (W,H,L) 처럼 **3D**이더라도, 필터 길이 L이 **입력 채널 길이와 같을 때, 2D 합성곱 형상**이 됨.
- 필터 (k,k,L) 적용 시 **출력은 (W,H)**.
- 대표 사례: **LeNet-5, VGG** (자세한 내용은 6장).

1×1 합성곱



- 3D 입력 (W,H,L) 에 필터 (1,1,L) 적용 → 출력 (W,H).
- 채널 수를 조정해 연산량 감소 효과.
- GoogLeNet 사례 언급(자세한 내용은 6장).

5.2 합성곱 신경망 맛보기

데이터셋: `fashion_mnist` (torchvision 내장 예제)

- 28×28 픽셀의 작은 이미지 7만 개로 구성(학습/테스트로 분할).
- 레이블은 0~9의 정수(운동화, 셔츠 등 10개 클래스).
 - 클래스 목록:
0: T-Shirt, 1: Trouser, 2: Pullover, 3: Dress, 4: Coat,
5: Sandal, 6: Shirt, 7: Sneaker, 8: Bag, 9: Ankle Boot

코드 5-1. 라이브러리 호출

```
import numpy as np
import matplotlib.pyplot as plt

import torch
import torch.nn as nn
from torch.autograd import Variable
```

```
import torch.nn.functional as F

import torchvision
import torchvision.transforms as transforms
from torch.utils.data import Dataset, DataLoader
```

코드 5-2. CPU 혹은 GPU 장치 확인

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
```

Note — GPU 사용

- 단일 GPU 예시:

```
device = torch.device("cuda:0" if torch.cuda.is_available() else "cpu")
model = Net()
model.to(device)
```

- 다중 GPU 예시(`nn.DataParallel` 사용):

```
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model = Net()
if torch.cuda.device_count() > 1:
    model = nn.DataParallel(model)
model.to(device)
```

코드 5-3. `fashion_mnist` 데이터셋 내려받기

```
train_dataset = torchvision.datasets.FashionMNIST(
    "../chap05/data", download=True,
    transform=transforms.Compose([transforms.ToTensor()])))
```

```
test_dataset = torchvision.datasets.FashionMNIST(
    "../chap05/data", download=True, train=False,
    transform=transforms.Compose([transforms.ToTensor()]))
```

- `torchvision.datasets` 는 `torch.utils.data.Dataset` 하위 클래스 모음.
- **파라미터**
 - ① 경로: 데이터 내려받을 위치
 - ② `download=True` : 해당 경로에 없으면 내려받음
 - ③ `transform=...` : 이미지를 텐서(0~1)로 변환

코드 5-4. `fashion_mnist` 데이터를 데이터로더에 전달

```
train_loader = torch.utils.data.DataLoader(
    train_dataset, batch_size=100)

test_loader = torch.utils.data.DataLoader(
    test_dataset, batch_size=100)
```

- `DataLoader(dataset, batch_size=...)`
 - ① **dataset**: 불러올 데이터셋 지정
 - ② **batch_size**: 배치 단위 크기(예: 100)
 - ③ `shuffle` 옵션으로 무작위 섞기 가능

코드 5-5. 분류에 사용될 클래스 정의 및 시각화

```
labels_map = {0:'T-Shirt', 1:'Trouser', 2:'Pullover', 3:'Dress', 4:'Coat',
              5:'Sandal', 6:'Shirt', 7:'Sneaker', 8:'Bag', 9:'Ankle Boot'}

fig = plt.figure(figsize=(8,8))
columns, rows = 4, 5
for i in range(1, columns*rows + 1):
    img_xy = np.random.randint(len(train_dataset))
    img = train_dataset[img_xy][0][0, :, :]
    fig.add_subplot(rows, columns, i)
```

```
plt.title(labels_map[train_dataset[img_xy][1]])
plt.axis('off')
plt.imshow(img, cmap='gray')
plt.show()
```

설명

- `np.random` 계열 함수 예시 제시
 - `np.random.randint(10)` : 0~10 미만 정수
 - `np.random.randint(1, 10)` : 1~9 정수
 - `np.random.rand(8)` : 0~1 균등분포 난수(1×8)
 - `np.random.randn(4,2)` 등: 정규분포 난수 (예시 출력 제시)
- 3차원 배열 생성·인덱싱 예시
 - `examp = np.arange(0, 100, 3)` → 1~99 범위 등차수열 생성 후 `examp.resize(6,4)` 로 6×4 배열
 - `examp[3]` : 4번째 행 전체
 - `examp[3, 3]` : (4행,4열) 원소
 - `examp[3][3]` : 위와 동일 결과
- 인덱싱 연결
 - `train_dataset[img_xy][0]` : 이미지 텐서
 - `[0, :, :]` : 0번째 채널(그레이스케일)의 전체 픽셀 선택
 - 위 인덱싱 의미를 책의 배열 예시로 유추하도록 설명됨.

배열 인덱싱 예시

- `examp = np.arange(0, 500, 3)` → `examp.resize(3, 5, 5)`
- 예: `examp[2][0][3] = 159`
- 위 인덱싱은 `train_dataset[img_xy][0][0,:,:]` 와 같이 채널/축을 지정해 요소 값을 가져오는 예로 연결해 이해하도록 제시됨.

♥ 그림 5-24 20개의 이미지 데이터를 시각적으로 표현



- 무작위 20장에 대해 클래스 라벨과 함께 그레이스케일로 출력됨. (랜덤 샘플이므로 결과 이미지는 실행마다 달라질 수 있음)

코드 5-6 심층 신경망 모델 생성

```
class FashionDNN(nn.Module):
    def __init__(self):
        super(FashionDNN, self).__init__()
        self.fc1 = nn.Linear(in_features=784, out_features=256)
        self.drop = nn.Dropout(0.25)
        self.fc2 = nn.Linear(in_features=256, out_features=128)
        self.fc3 = nn.Linear(in_features=128, out_features=10)

    def forward(self, input_data):
        out = input_data.view(-1, 784)
        out = F.relu(self.fc1(out))
        out = self.drop(out)
        out = F.relu(self.fc2(out))
        out = self.fc3(out)
        return out
```

① 클래스는 `torch.nn.Module` 을 상속. `__init__()` 은 객체 초기화, `super(...).__init__()` 은 부모 클래스 초기화 호출.

Note — 객체 / 클래스와 함수

- 객체지향 프로그래밍 개념(클래스/객체)을 간단 예와 함께 설명.
- 함수 예와 클래스 예(`Calc`)를 통해 **함수 재사용**과 **클래스 상태 보관** 차이를 비교.

② `nn.Linear(in_features=784, out_features=256)`

- `in_features` : 입력 크기, `out_features` : 출력 크기.
- ③ `torch.nn.Dropout(p)` : 비율 `p` 만큼 텐서 값을 0으로 만들어 학습 시 과적합을 완화.
- ④ `forward()` : 순전파 함수. (학습 데이터 입력 → 모델 내 연산 → 출력)
- ⑤ `view` : 텐서 크기(shape) 변경(NumPy `reshape` 과 유사). `input_data.view(-1, 784)` 는 배치×784로 평탄화.
- ⑥ 활성화 함수 지정 두 방법
- `F.relu(...)` : `nn.functional` 안에서 **함수 호출**
- `nn.ReLU()` : **모듈로 정의** 후 계층처럼 사용

nn.xx 와 nn.functional.xx 예·비교

- 모듈 방식:

```
conv = nn.Conv2d(in_channels=3, out_channels=64, kernel_size=3, padding=1)
outputs = conv(inputs)
layer = nn.Conv2d(1, 1, 3)
```

- 함수 방식:

```
outputs = F.conv2d(inputs, weight, bias, padding=1)
```

구분	nn.xx	nn.functional.xx
형태	<code>nn.Conv2d</code> : 클래스 <code>nn.Module</code> 클래스를 상속받아 사용	<code>nn.functional.conv2d</code> : 함수 <code>def function (input)</code> 으로 정의된 순수한 함수
호출 방법	먼저 하이퍼파라미터를 전달한 후 함수 호출을 통해 데이터 전달	함수를 호출할 때 하이퍼파라미터, 데이터 전달
위치	<code>nn.Sequential</code> 내에 위치	<code>nn.Sequential</code> 에 위치할 수 없음

구분	nn.xx	nn.functional.xx
파라미터	파라미터를 새로 정의할 필요 없음	가중치를 수동으로 전달해야 할 때마다 자체 가중치를 정의

코드 5-7 심층 신경망에서 필요한 파라미터 정의

```
learning_rate = 0.001
model = FashionDNN()
model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
print(model)
```

- 옵티마이저: **Adam** 사용, 학습률 `lr=0.001`.

출력:

```
FashionDNN(
  (fc1): Linear(in_features=784, out_features=256, bias=True)
  (drop): Dropout(p=0.25, inplace=False)
  (fc2): Linear(in_features=256, out_features=128, bias=True)
  (fc3): Linear(in_features=128, out_features=10, bias=True)
)
```

코드 5-8 심층 신경망을 이용한 모델 학습

```
num_epochs = 5
count = 0

loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
```

```

labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)
            predictions_list.append(predictions)
            correct += (predictions == labels).sum()
            total += len(labels)

        accuracy = correct * 100 / total
        loss_list.append(loss.data)
        iteration_list.append(count)
        accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}".format(
            count, loss.data, accuracy))

```

- ① 리스트 생성·추가(`append`) 사용.
- ② `for x, y in train:` 형태의 **튜플 언패킹** 예시 제시(그림 5-26).
- ③ 모델·데이터는 동일 장치(CPU/GPU)에서 처리.
- ④ **Autograd**는 순전파 계산 기록을 남기고, 역전파 시 자동으로 미분값 계산(코드에서는 `Variable` 사용).
- ⑤ 분류 정확도 계산:

```
classification accuracy = (correct predictions / total predictions) * 100
error rate = (1 - (correct predictions / total predictions)) * 100
```

학습 로그

```
Iteration: 500, Loss: 0.5627..., Accuracy: 82.79...
Iteration: 1000, Loss: 0.4662..., Accuracy: 84.40...
Iteration: 1500, Loss: 0.3302..., Accuracy: 84.41...
Iteration: 2000, Loss: 0.3137..., Accuracy: 85.95...
Iteration: 2500, Loss: 0.2844..., Accuracy: 86.26...
Iteration: 3000, Loss: 0.3125..., Accuracy: 86.48...
```

- 최종 약 **86%** 정확도.

코드 5-9 합성곱 네트워크 생성

```
class FashionCNN(nn.Module):
    def __init__(self):
        super(FashionCNN, self).__init__()
        self.layer1 = nn.Sequential(
            nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1),
            nn.BatchNorm2d(32),
            nn.ReLU(),
            nn.MaxPool2d(kernel_size=2, stride=2)
        )
        self.layer2 = nn.Sequential(
            nn.Conv2d(in_channels=32, out_channels=64, kernel_size=3),
```

```

        nn.BatchNorm2d(64),
        nn.ReLU(),
        nn.MaxPool2d(2)
    )
    self.fc1 = nn.Linear(in_features=64*6*6, out_features=600)
    self.drop = nn.Dropout2d(0.25)
    self.fc2 = nn.Linear(in_features=600, out_features=120)
    self.fc3 = nn.Linear(in_features=120, out_features=10)

    def forward(self, x):
        out = self.layer1(x)
        out = self.layer2(out)
        out = out.view(out.size(0), -1)
        out = self.fc1(out)
        out = self.drop(out)
        out = self.fc2(out)
        out = self.fc3(out)
        return out

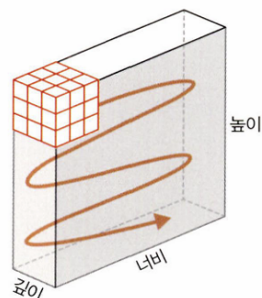
```

설명

- ① `nn.Sequential` 로 여러 계층을 순차 연결.
- ② 합성곱층: `nn.Conv2d(in_channels=..., out_channels=..., kernel_size=..., padding=...)`
 - `in_channels` : 입력 채널 수(흑백=1, RGB=3 등).
 - 2D 합성곱에서 채널은 `w×h×c` 중 **c(깊이)** 에 해당.

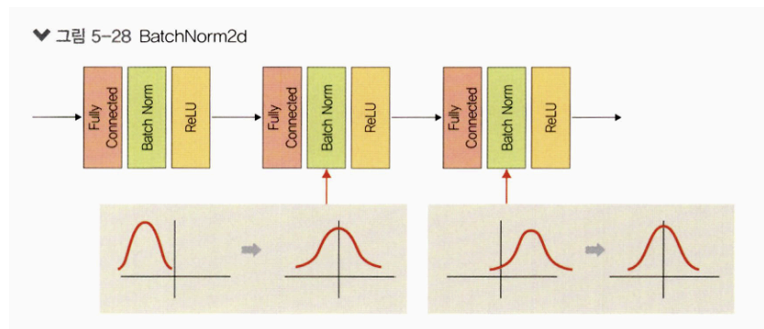
합성곱 계층 파라미터 설명

▼ 그림 5-27 채널



- 채널(깊이), 너비, 높이로 표시됨. 커널이 스트라이드 경로를 따라 이동함.
- **out_channels**: 출력 채널의 수를 의미한다.
- **kernel_size**: 커널 크기를 의미하며 논문에 따라 필터라고도 한다. 커널은 이미지 특성을 찾아내기 위한 공용 파라미터이며, CNN에서 학습 대상은 필터 파라미터가 된다. 커널은 입력 데이터를 스트라이드 간격으로 순회하면서 합성곱을 계산한다.
 - 예: `kernel_size=3` → (3, 3). 직사각형을 사용하고 싶다면 (3, 5)처럼 지정한다.
- **padding**: 패딩 크기를 의미하는 것으로 출력 크기를 조정하기 위해 입력 데이터 주위에 0을 채운다. 패딩 값이 클수록 출력 크기도 커진다.

③ **BatchNorm2d**: 학습 과정에서 각 배치 단위별로 데이터가 다양한 분포를 가지더라도 평균과 분산을 이용하여 정규화하는 것을 의미한다.



- 배치 단위나 계층에 따라 입력값 분포가 모두 다르지만 정규화를 통해 분포를 가우시안 형태로 만든다. (평균은 0, 표준편차는 1로 데이터의 분포가 조정됨)
- ④ **MaxPool2d**: 이미지 크기를 축소시키는 용도로 사용한다. 풀링 계층은 합성곱층의 출력 데이터를 입력으로 받아서 출력 데이터(activation map)의 크기를 줄이거나 특정 데이터를 강조하는 용도로 사용된다. 풀링 계층을 처리하는 방법으로는 **최대 풀링(max pooling)**, **평균 풀링(average pooling)**, **최소 풀링(min pooling)** 이 있으며, 이때 사용하는 파라미터는 다음과 같다.
- **kernel_size**: $m \times n$ 행렬로 구성된 가중치
 - **stride**: 입력 데이터에 커널(필터)을 적용할 때 이동할 간격. 스트라이드 값이 커지면 출력 크기는 작아진다.
- ⑤ 분류를 분류하기 위해서는 이미지 형태의 데이터를 배열 형태로 변환하여 작업해야 한다. 이때 **Conv2d**에서 사용하는 하이퍼파라미터 값들에 따라 **출력 크기(output size)**가 달라진다(즉, 패딩과 스트라이드의 값에 따라 출력 크기가 달라짐). 이렇게 줄어든 출력 크기는 최종적으로 분류를 담당하는 **완전연결층**으로 전달된다.

Conv2d 계층에서의 출력 크기 구하는 공식

$$\text{출력크기} = (W - F + 2P)/S + 1$$

- W: 입력 데이터의 크기(input_volume_size)
- F: 커널 크기(kernel_size)
- P: 패딩 크기(padding_size)
- S: 스트라이드(strides)

예를 들어 첫 번째 Conv2d 계층은 다음과 같다.

```
nn.Conv2d(in_channels=1, out_channels=32, kernel_size=3, padding=1)
```

따라서 출력 크기는 다음과 같이 계산할 수 있다.

$$(784 - 3 + (2 * 1))/1 + 1 = 784$$

- (fashion_mnist의 입력 데이터 크기는 784이며, stride가 명시되어 있지 않다면 stride 기본값은 (1,1).)
- 계산 결과를 적용하면 **출력의 형태는 [32, 784, 784]** 가 된다.

MaxPool2d 계층에서의 출력 크기 구하는 공식

$$\text{출력크기} = IF/F$$

- IF: 입력 필터의 크기(input_filter_size, 또한 바로 앞의 Conv2d의 출력 크기)
- F: 커널 크기(kernel_size)

예: 첫 번째 MaxPool2d 계층

```
nn.MaxPool2d(kernel_size=2, stride=2)
```

따라서 출력 크기는 다음과 같이 계산할 수 있다.

$$784/2 = 392$$

- 계산 결과를 적용한 **출력의 형태는 [32, 392, 392]** 이다.
 - `out_features` : 출력 데이터의 크기를 의미합니다.
- ⑥ 합성곱층에서 완전연결층으로 변경되기 때문에 데이터의 형태를 **1차원으로 바꾸어 준다.**
- `out.size(0)` 은 결국 **100**을 의미한다.

- 따라서 (100, ?) 크기의 텐서로 변경하겠다는 의미.
- `out.view(out.size(0), -1)` 에서 1 은 **행(row)** 의 수는 정확히 알고 있지만 **열(column)** 의 수를 알지 못할 때 사용.

코드 5-10 합성곱 네트워크를 위한 파라미터 정의

```
learning_rate = 0.001
model = FashionCNN()

model.to(device)

criterion = nn.CrossEntropyLoss()
optimizer = torch.optim.Adam(model.parameters(), lr=learning_rate)
print(model)
```

모델 구조 출력

```
FashionCNN(
  (layer1): Sequential(
    (0): Conv2d(1, 32, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1))
    (1): BatchNorm2d(32, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (layer2): Sequential(
    (0): Conv2d(32, 64, kernel_size=(3, 3), stride=(1, 1))
    (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
    (2): ReLU()
    (3): MaxPool2d(kernel_size=2, stride=2, padding=0, dilation=1, ceil_mode=False)
  )
  (fc1): Linear(in_features=2304, out_features=600, bias=True)
  (drop): Dropout2d(p=0.25, inplace=False)
  (fc2): Linear(in_features=600, out_features=120, bias=True)
```

```
(fc3): Linear(in_features=120, out_features=10, bias=True)
)
```

코드 5-11 모델 학습 및 성능 평가

```
num_epochs = 5
count = 0
loss_list = []
iteration_list = []
accuracy_list = []

predictions_list = []
labels_list = []

for epoch in range(num_epochs):
    for images, labels in train_loader:
        images, labels = images.to(device), labels.to(device)
        train = Variable(images.view(100, 1, 28, 28))
        labels = Variable(labels)

        outputs = model(train)
        loss = criterion(outputs, labels)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
        count += 1

    if not (count % 50):
        total = 0
        correct = 0
        for images, labels in test_loader:
            images, labels = images.to(device), labels.to(device)
            labels_list.append(labels)
            test = Variable(images.view(100, 1, 28, 28))
            outputs = model(test)
            predictions = torch.max(outputs, 1)[1].to(device)
```



```

        predictions_list.append(predictions)
        correct += (predictions == labels).sum()
        total += len(labels)

    accuracy = correct * 100 / total
    loss_list.append(loss.data)
    iteration_list.append(count)
    accuracy_list.append(accuracy)

    if not (count % 500):
        print("Iteration: {}, Loss: {}, Accuracy: {}".format(
            count, loss.data, accuracy))

```

모델 훈련 결과

```

Iteration: 500, Loss: 0.4392..., Accuracy: 87.87...
Iteration: 1000, Loss: 0.3259..., Accuracy: 88.09...
Iteration: 1500, Loss: 0.3011..., Accuracy: 88.39...
Iteration: 2000, Loss: 0.1754..., Accuracy: 89.62...
Iteration: 2500, Loss: 0.1377..., Accuracy: 89.56...
Iteration: 3000, Loss: 0.2483..., Accuracy: 88.94...

```

마무리

- 심층 신경망과 비교하여 정확도가 약간 높다.
- 심층 신경망과 별 차이가 없기 때문에 좀 더 간편한 심층 신경망만 사용해도 무난할 것 같지만, 실제로 이미지 데이터가 많아지면 **단순 심층 신경망으로는 정확한 특성 추출 및 분류가 불가능**하므로 **합성곱 신경망을 생성**할 수 있도록 학습해야 한다.